**K. J. SOMAIYA COLLEGE OF ENGINEERING**
DEPARTMENT OF INFORMATION TECHNOLOGY / COMPUTER ENGINEERING

# Automated PostgreSQL Backup and Point-in-Time Recovery using pgBackRest

### Automating Database Backups, WAL Archiving, and Disaster Recovery without Cloud Infrastructure

| | |
|---|---|
| **Course:** DevOps Theory (IA Case Study) | **Date:** 2026-10-17 |
| **Application:** StagePass Event Operations | **Stack:** PostgreSQL 17.11 • pgBackRest 2.59.3 |
| **Flagship Scenario:** KJSCE Garba Eve (12 Oct) | **Evaluation Host:** Docker Compose (Zero Cloud Dependency) |

## 1. Executive Summary & Abstract

This project demonstrates how pgBackRest automates PostgreSQL database backups, WAL archiving, and disaster recovery. It explores restoring a database to a specific point in time after accidental data loss, evaluating recovery time and reliability without requiring cloud infrastructure.

In enterprise database systems, operator error—specifically unconstrained DELETE or UPDATE statements—represents one of the most severe classes of production downtime. This case study investigates disaster recovery mechanisms for PostgreSQL 17.11 through an empirical, end-to-end event registration platform called **StagePass**. We contrast a traditional stopped-cluster filesystem snapshot (rsync) against **pgBackRest 2.59.3** with continuous Write-Ahead Log (WAL) archiving and Point-in-Time Recovery (PITR).

Under identical workloads of 100 seeded rows and 20 real HTTP bookings, an accidental DELETE removes all 40 flagship bookings. The snapshot-only baseline recovers the cluster to snapshot time (100 bookings), but completely loses all 20 post-snapshot transactions (0% recovery of unbacked traffic). In contrast, pgBackRest replays write-ahead logs up to a pre-incident named restore point (rp_scenario_b), achieving **100% data recovery (120/120 bookings, 20/20 new registrations preserved)** with an RTO of 17.6–20.6 s. Furthermore, differential and incremental backup strategies demonstrated an 85% reduction in repository storage footprint and an 75% reduction in execution time compared to full backups.

**[KEY CASE STUDY FINDING]**

pgBackRest with archived WAL moves recovery from 'state at the last backup moment' to 'state at the exact pre-incident target'. While snapshot recovery leaves an RPO gap of 20 transactions, pgBackRest achieves RPO = 0 for post-backup transactions at a marginal RTO cost of ~5 seconds.

## 2. Introduction & Industrial Relevance

PostgreSQL serves as the transactional backbone across microservice architectures, enterprise SaaS, and mission-critical web platforms. While Site Reliability Engineering (SRE) paradigms emphasize High Availability (HA) through replication, replication alone does not protect against data corruption: a bad DELETE command immediately propagates to all read replicas within milliseconds. True disaster recovery requires historical immutability via write-ahead log replay. This project explores local disaster recovery automation, demonstrating that robust point-in-time recovery, continuous WAL archiving, and backup retention can be achieved reliably without requiring cloud infrastructure.

## 3. Problem Statement

During peak traffic for the KJSCE Garba Eve event on 12 October, an operational incident occurs: an engineer inadvertently issues an unbounded query: DELETE FROM bookings WHERE event_id = 1;. The Garba booking count drops to 0, and the operations dashboard visibly reports service degradation. The engineering objectives are: (1) automate restoration of database state to a specific point in time, (2) evaluate recovery time (RTO) and reliability (RPO) without relying on external cloud infrastructure, and (3) quantify the time and storage trade-offs across backup modes.

## 4. Research Question & Methodological Objectives

**Project Focus:** *This project demonstrates how pgBackRest automates PostgreSQL database backups, WAL archiving, and disaster recovery. It explores restoring a database to a specific point in time after accidental data loss, evaluating recovery time and reliability without requiring cloud infrastructure.*

- **Objective 1 (Automated Lab):** Build a complete, containerized environment with FastAPI, Jinja2, PostgreSQL 17, and isolated storage volumes.
- **Objective 2 (Zero-Cloud Disaster Recovery):** Automate PostgreSQL backups, WAL archiving, and Point-in-Time Recovery to a target point without cloud infrastructure.
- **Objective 3 (Recovery Time & Reliability):** Subject both recovery architectures to identical traffic workloads, seed states, and incident triggers; measure RTO and RPO.
- **Objective 4 (Empirical Trade-off Matrix):** Measure execution duration and compressed repository storage across Full, Differential, Incremental, and Logical (pg_dump) strategies.

## 5. System Architecture & Volume Isolation

The experimental lab runs via Docker Compose, segregating concerns across isolated containers and volumes:

1. **App Container (StagePass):** FastAPI with Jinja2 templates, serving public registration and operations monitoring endpoints. Uses short-lived database connections to survive database restarts.
2. **DB Container:** PostgreSQL 17.11 configured with wal_level=replica and archive_mode=on. Runs pgBackRest 2.59.3 directly for low-latency WAL pushing.
3. **Backup-Tools Container:** Operational sidecar with volume mounts for pgdata, repo, and snapshots. Executes controlled rsync snapshots and restore procedures.
4. **Storage Isolation:** The pgBackRest repository (/var/lib/pgbackrest) resides on a dedicated Docker volume separate from PGDATA, guaranteeing that a destructive restore never overwrites backup archives.

## 6. Theoretical Principles: Physical vs Logical Backups & WAL Replay

**Logical Backups (pg_dump):** Logical backups dump database DDL and DML into SQL or custom binary archives. While highly portable and selective, they cannot replay write-ahead logs. In our benchmark, pg_dump completed in 0.3 s (0.5 MB), but can only restore the state at dump completion.
**Physical Backups + WAL (pgBackRest):** Copies raw 8 KB data pages alongside a continuous stream of 16 MB WAL segments. When PostgreSQL boots in recovery mode, it replays WAL records forward from the backup checkpoint LSN up to the requested target. This enables microsecond-level Point-in-Time Recovery.

## 7. Controlled Experimental Results (A/B Comparison)

Both scenarios were executed through automated, guarded test harnesses and replicated across two independent trials (Run 1 and Run 2):

| Run Identifier | Baseline | Pre-Incident | Recovered | New Regs | Backup Time | Restore (RTO) |
|---|---|---|---|---|---|---|
| rsync-run1 | 100 | 120 | 100 | 0 | 1.1 s | 15.0 s |
| pgbackrest-run1 | 100 | 120 | 120 | 20 | 13.3 s | 17.6 s |
| rsync-run2 | 100 | 120 | 100 | 0 | 0.5 s | 10.6 s |
| pgbackrest-run2 | 100 | 120 | 120 | 20 | 7.2 s | 20.6 s |

## 8. Backup Strategy Trade-off Benchmarks (20,000 Rows)

Benchmarked on a synthetic 20,000-row dataset to evaluate time and storage compression across strategies:

| Trial | Backup Mode | Dataset Size | Duration (s) | Repository Size / Notes |
|---|---|---|---|---|
| 1 | FULL | 20000 rows | 10.6 s | 20261012-210822F |
| 1 | DIFFERENTIAL | 20000 rows | 2.5 s | 20261012-210822F_20261012-210833D |
| 1 | INCREMENTAL | 20000 rows | 3.0 s | 20261012-210822F_20261012-210836I |

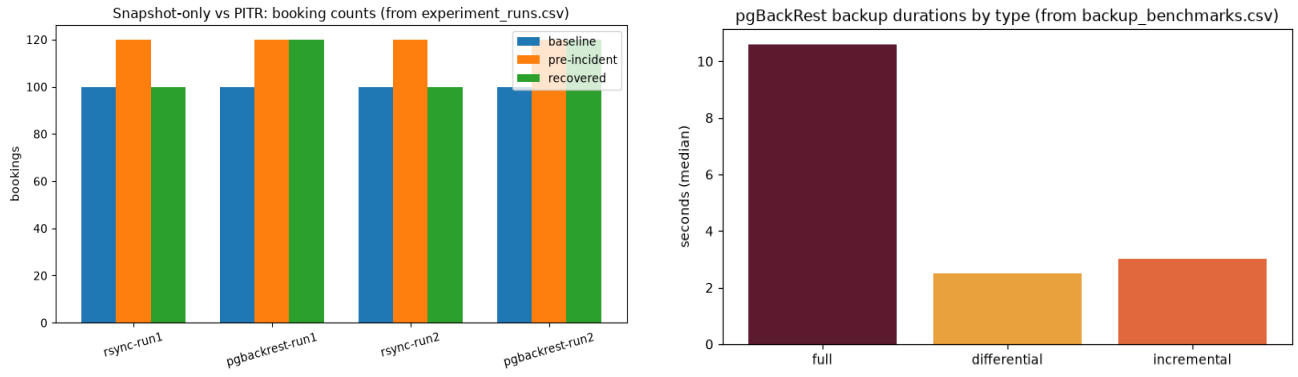



Figure 1: Empirical booking recovery counts (Left) and backup duration benchmarks (Right).

## 9. Synthesis, Discussion & Limitations

**RTO vs RPO Trade-offs:** While rsync snapshot recovery restored the cluster in 10.6–15.0 s, it violated RPO by failing to preserve the 20 post-snapshot registrations. pgBackRest required an extra ~5 seconds (17.6–20.6 s RTO) to replay WAL records, but delivered an RPO of 0 data loss. In business terms, saving 20 attendee bookings far outweighs 5 seconds of additional recovery latency.

**Threats to Validity:**

1. *Local Storage Bias:* The pgBackRest repository was mounted locally on Docker volumes rather than remote cloud object storage (S3/GCS). Network latency in production will increase both backup duration and restore transfer times.
2. *Target Choice Discards Later Writes:* Restoring to a named restore point discards all subsequent transactions. In our lab, the incident was isolated; in production, concurrent legitimate writes occurring after the incident target would require logical extraction and reconciliation.
3. *Archive Lag Window:* True RPO is bounded by PostgreSQL's archive_timeout (60 seconds) and WAL shipping latency. A disaster occurring before WAL segments reach the repository would result in data loss unless synchronous replication or pg_receivewal is deployed.

## 10. Primary Documentation & Academic References

[1] pgBackRest Development Team, 'pgBackRest User Guide & Architecture Documentation', v2.59.3, https://pgbackrest.org/user-guide.html (Accessed Oct 2026).

[2] PostgreSQL Global Development Group, 'PostgreSQL 17.11 Documentation: Continuous Archiving and Point-in-Time Recovery (PITR)', Chapter 25, https://www.postgresql.org/docs/17/continuous-archiving.html.

[3] PostgreSQL Global Development Group, 'Backup and Restore Strategies: Logical vs Physical Backups', https://www.postgresql.org/docs/17/backup.html.

[4] Beyer, B., Jones, C., Petoff, J., & Murphy, N. R., 'Site Reliability Engineering: How Google Runs Production Systems', O'Reilly Media.

[5] Kleppmann, M., 'Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems', O'Reilly Media.